

# TRAN HUY VIET

Ha Noi • [teiwiet33@gmail.com](mailto:teiwiet33@gmail.com) • +84975972419

[github.com/teiwiet](https://github.com/teiwiet) • [teiwiet.github.io](https://teiwiet.github.io)

Security researcher focused on firmware and embedded systems security. Hands-on experience through self-directed research in firmware reverse engineering, vulnerability discovery, and exploit development on MIPS/ARM targets, backed by a strong foundation in C/C++, Rust, and Linux internals. Passionate about building offensive tooling and contributing to vendor security through responsible disclosure.

## WORK EXPERIENCE

### Network Maintenance Engineer (Intern)

Jun 2025 - Nov 2025

#### FPT-Telecom

Ha Noi

- Supported troubleshooting of routers & switches, handling customer network issues.
- Performed on-site device configuration, firmware upgrades, and network diagnostics for residential customers.

## EDUCATION

### Bachelor of Engineering in Telecommunications Engineering

Sep 2020 - Nov 2025

Posts and Telecommunications Institute of Technology (PTIT)

## SKILLS

**Languages:** C, C++, Rust, Assembly

**English:** TOEIC 865

**Security:** Firmware reverse engineering, vulnerability discovery, exploit development (MIPS/ARM), responsible disclosure

**Reverse Engineering & Analysis:** Ghidra, Binwalk, GDB, QEMU (user-mode & system), ELF analysis

**Embedded & OS Internals:** STM32, Raspberry Pi, Linux Kernel, Bare-metal programming, eBPF/libbpf

## PROJECTS

### Firmware Exploitation Research

- Performed end-to-end security research on multiple MIPS-based routers — from hardware-level firmware extraction through emulation, analysis, and exploitation.
- Acquired firmware at the hardware level: after probing the PCB for UART and getting only garbled output, dumped firmware directly from SPI flash with a CH341A programmer, rather than relying on vendor-supplied binaries.
- Unpacked filesystems with binwalk and emulated services in QEMU (user- & system-mode) for dynamic analysis; recovered hardcoded credentials and private keys via static analysis.
- Reproduced and built working exploits for two known vulnerabilities: a command injection achieving arbitrary command execution (validated under GDB), and a stack buffer overflow in TP-Link WR740N httpd, with exploitability assessed on a no-ASLR MIPS target.
- Published writeups: [teiwiet.github.io/writeup/netgear.html](https://teiwiet.github.io/writeup/netgear.html) | [teiwiet.github.io/writeup/tplink.html](https://teiwiet.github.io/writeup/tplink.html)

### Binary Fuzzer

- Built a mutation-based firmware fuzzer in Rust targeting MIPS/ARM binaries via QEMU user-mode emulation
- Implemented multi-strategy mutator (bit flip, byte flip, format string injection, path traversal, null bytes) with crash-driven corpus growth
- Developed ELF static analyzer to score and prioritize high-risk binaries by dangerous function presence (strcpy, system, execve) and input entry points (recv, scanf)
- Designed modular harness supporting both native and QEMU execution modes with configurable timeout handling

### kprowl — eBPF Dynamic Analysis Sandbox

- Built an eBPF-based dynamic analysis sandbox (C + libbpf) that traces an untrusted binary's runtime behavior — process execution, file access, and network connections — without modifying the target or using a heavyweight VM.
- Implemented process-tree scoping via a BPF hash map seeded before execve (pipe-synchronized fork/exec) and sched\_process\_fork propagation, isolating the target and its descendants from system-wide noise.
- Attached BPF programs to kernel tracepoints (execve, openat, connect, fork/exit), streaming events to user space through a BPF ring buffer with CO-RE for cross-kernel portability.
- Added structured JSON output, loader/runtime noise filtering, and a process-tree/timeline report.

### Firmware Security Analyzer

- Developed an automated CLI firmware security analysis pipeline for IoT firmware.
- Implemented firmware extraction and architecture detection (MIPS/ARM) using binwalk and ELF inspection.
- Built static scanners to detect hardcoded credentials, private keys, and unsafe functions in extracted filesystems.
- Implemented heuristics to identify command injection patterns and dangerous system calls.
- Detected embedded web servers and CGI endpoints, enabling identification of potential web attack surfaces.

### Operating System Development (Ongoing)

- Developing a hobby x86 operating system in C and Assembly.
- Implemented bootloader interaction and early kernel initialization.
- Built simple framebuffer-based graphics modules for basic rendering.
- Exploring low-level system components and kernel architecture.